



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 – SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

TUTORIAL 3:

AI ALGORITHM

LECTURER: DR. ARYATI BINTI BAKRI

GROUP 2

STUDENT NAME	MATRIC NO
CHAU YING JIA	A23CS0213
LAU YEE WEN	A23CS0099
POH LOK YEE	A23CS0262

Table of Content

1.0 Introduction.....	3
2.0 Basic Concepts of Convolutional Neural Network (CNN).....	4
2.1 Image Classification.....	4
2.2 Convolutional Neural Network (CNN).....	4
2.3 Evaluation of Convolutional Neural Network.....	5
3.0 Steps Hands-on in Python.....	7
4.0 Model Evaluation and Comparative Analysis.....	17
4.1 Limitations of the Baseline CNN.....	17
4.1.1 Shallow and Small Architecture.....	17
4.1.2 Overfitting.....	17
4.1.3 Absence of Data Augmentation.....	17
4.2 Strategies for Model Enhancement.....	18
4.2.1 Architectural Expansion and Dense Layer Tuning.....	18
4.2.2 Batch Normalization and Dropout.....	19
4.2.3 Data Augmentation.....	19
4.3 Performance Evaluation of CNN and New CNN.....	20
4.4 Result and Discussion.....	23
5.0 Reflection.....	24

1.0 Introduction

Image classification is considered as one of the major applications in the domain of Artificial Intelligence and Computer Vision. In general, image classification refers to the process of classifying and categorizing images into predefined classes based on their visual features. The system will learn patterns from a large number of training images and then predict the class of new images based on their characteristics. Image classification has already been utilized in various applications like facial recognition, medical classification, self-driving cars, security cameras or object recognition.

Traditional image classification methods use feature extraction manually where humans need to define the image features themselves like edges, shapes or textures before classification can occur. However, this traditional machine learning method requires a lot of time to be executed and does not perform very accurately on complex and large image datasets. Nowadays, Convolutional Neural Network (CNN) has become one of the most popular and effective techniques for image classification tasks. CNNs are able to automatically detect features contained in images during the training process through multiple layers such as convolution layers, pooling layers and fully connected layers. It achieves higher accuracy and with better performance as it learns image features automatically without doing classification manually.

In this tutorial, a CNN model was developed using Python by following the tutorial video “Image Classification Using CNN”. Jupyter Notebook is used for writing code here. This tutorial demonstrates the complete process of image classification including data processing, model training, evaluation and prediction generation. The main goal of this project is to let students familiarize with the image classification process and to provide students knowledge of CNN algorithms. It also gave practical experience and work for students to create models using Python.

2.0 Basic Concepts of Convolutional Neural Network (CNN)

2.1 Image Classification

Image classification is one of the computer vision techniques that enables an artificial intelligence system to identify and classify images according to pre-defined categories. This model will first analyze the image of its visual patterns then predict the best label for it based on the training dataset. For example, an image classification system can determine whether an image contains a cat, dog or dolphin.

In fact, the process of image classification involves a number of steps such as image pre-processing, feature detection, model training and predictions. For an AI algorithm like image classification using CNN, the feature detection is performed automatically. This could significantly improve classification results and decrease human works.

Due to the rapid growth of digital image data, image classification has become incredibly important to make images easily get categorized. There are more and more companies trying to use or using image classification systems to automate tasks to improve operational efficiency of work. The most common example is in healthcare, image classification is used to detect diseases from medical scans. On the other hand, image classification helps retail industries to identify all kinds of products automatically while in transportation, it could support autonomous driving systems through object recognition.

2.2 Convolutional Neural Network (CNN)

Convolutional Neural Network(CNN) is a type of deep learning neural network that has been explicitly developed for analyzing images and performing tasks related to computer vision. CNN is a very efficient algorithm as it could extract spatial and visual features from the train data automatically.

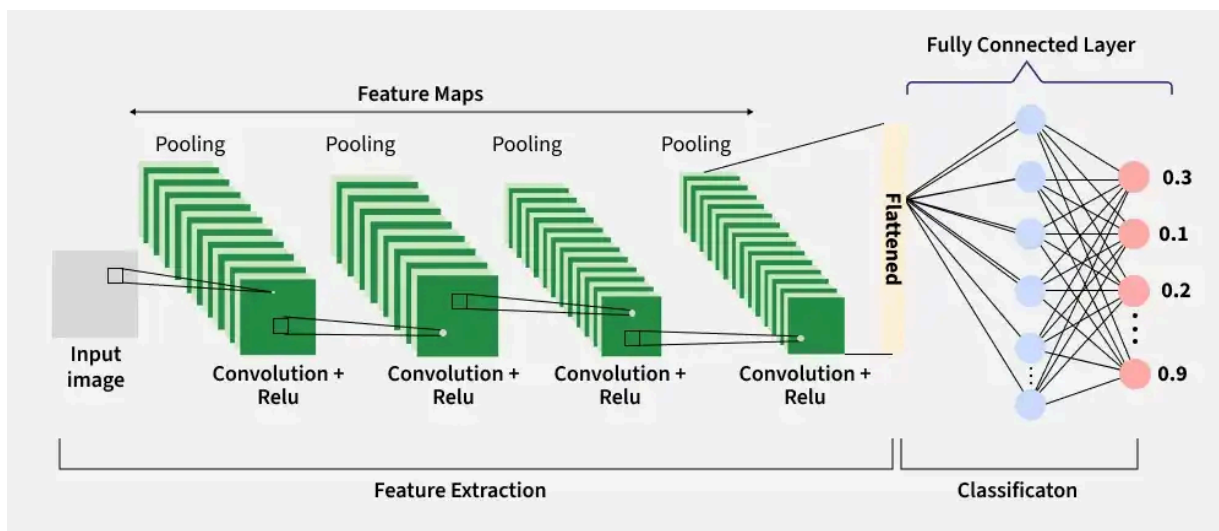


Figure 2.1 A Convolutional Neural Network (CNN)

Figure 2.1 shows the architecture of a CNN. It consists of several important layers including:

a. Convolutional Layer

The convolution layer is considered as the backbone of a Convolutional Neural Network. Filters are applied on the input image(training data) to detect important features. Different filters will detect the different aspects of the image like edges, texture, shape and patterns.

b. Activation Function

After the convolutional layer, an activation function is used to add a non-linearity concept to the CNN model. For instance, Rectified Linear Unit (ReLU) enables the model to learn complex, unpredictable and non-linear patterns.

c. Pooling Layer

Pooling Layer is responsible for reducing the size of the detected features by the convolutional layer. This process makes the computation easier and could reduce overfitting. Max Pooling is the most common type of pooling method used. It will select the maximum value from a feature region to reduce size.

d. Fully Connected Layer

The fully connected layer will combine all the learned features from the previous layer and perform the final classification process. Then, the extracted features will be combined to determine the predicted class label of the image.

e. Output Layer

The Output Layer provides the final result of the model. For the classification problem, the Softmax function is used for determining the probabilities.

2.3 Evaluation of Convolutional Neural Network

During image classification, the performance of Convolutional Neural Network(CNN) is evaluated using several evaluation metrics. Measuring accuracy alone is not enough. These metrics are used to gauge CNN's ability to accurately identify images and make accurate predictions on unseen data.

Accuracy is one of the most important metrics. It measures the percentage of the images that are correctly classified by the CNN model. The greater the accuracy, the better the model will perform. The CNN model in this tutorial performed around 69.48% accuracy on the testing set, showing that the CNN has a good accuracy on the majority of images being correctly classified. The CNN was able to learn features of images like edges, texture and shapes automatically which made it slightly better than the ANN model that was able to attain about 48% accuracy only.

Another metric that could be determined is loss. Loss measures how far the model's predictions are from the actual answers. During training of data, the objective of the CNN is to reduce the loss value as much as possible. A smaller loss value means that the predictions are more accurate. In this tutorial, the training

loss gradually decreased throughout the epochs, showing that the CNN successfully improves its learning performance over time.

Besides accuracy and loss, precision is also used to evaluate CNN performance. Precision measures how many predicted images for a specific class are actually correct. High precision means the model makes fewer incorrect predictions for that particular category. For example, if CNN predicts an image is a cat, precision determines how often the prediction is truly a cat.

Recall is another useful metric that measures how many actual images from a class are successfully detected by the model. Therefore, a high recall value implies that a CNN model can recognize most of the images that belong to a certain class. This metric becomes critical when there are classes in the CIFAR-10 set containing similar objects that are very hard to differentiate.

Finally, the F1-score can be used to evaluate the efficiency of CNN models for classifying performance. The F1-score is computed using both precision and recall values. The higher the F1 score the better the overall classification is achieved.

In summary, all the metrics mentioned above allow for getting a clearer picture of the efficiency of CNN models for image classification. These metrics show what aspects require improvement in order to get more accurate classification results. Other than that, they can also adjust the model further to enhance the classification accuracy and mitigate overfitting.

3.0 Steps Hands-on in Python

1. Dataset Loading

```
# 1. Import libraries
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np
```

Figure 3.1: Importing libraries for CNN image classification.

Based on Figure 3.1, several important Python libraries were imported to support the implementation of the CNN model. TensorFlow and Keras were used to build and train the deep learning model, NumPy was used for numerical operations, and Matplotlib was used for data visualization and displaying sample images.

2. Loading CIFAR-10 Dataset

```
# 2. Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
```

Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 3s 0us/step

Figure 3.2: Loading the CIFAR-10 dataset and defining image classes.

As shown in Figure 3.2, the CIFAR-10 dataset was loaded using the `datasets.cifar10.load_data()` function. The dataset consists of 60,000 color images categorized into 10 different classes such as airplane, automobile, bird, cat, dog, and truck. The dataset was automatically divided into training and testing datasets.

3. Dataset Shape Checking

```
# 3. Check dataset shape
print("x_train shape:", x_train.shape)
print("y_train shape:", y_train.shape)
print("x_test shape:", x_test.shape)
print("y_test shape:", y_test.shape)
```

```
x_train shape: (50000, 32, 32, 3)
y_train shape: (50000, 1)
x_test shape: (10000, 32, 32, 3)
y_test shape: (10000, 1)
```

Figure 3.3: Checking the shape of training and testing datasets.

Based on Figure 3.3, the training dataset contains 50,000 images while the testing dataset contains 10,000 images. Each image has a resolution of 32×32 pixels with 3 color channels (RGB). This step is important to verify that the dataset is loaded correctly before model training begins.

4. Reshaping the training labels

```
y_train[:5]
array([[6],
       [9],
       [9],
       [4],
       [1]], dtype=uint8)

y_train = y_train.reshape(-1,)
y_train[:5]
array([6, 9, 9, 4, 1], dtype=uint8)
```

Figure 3.4: Reshaping the training and testing labels into one-dimensional arrays.

Based on Figure 3.4, the training and testing labels were reshaped into one-dimensional arrays using the `reshape(-1,)` function. Originally, the labels were stored in a two-dimensional format such as (50000,1). After reshaping, the labels became a one-dimensional array, which simplifies the processing of class labels during model training and visualization.

The output also shows several sample labels from the training dataset, including values such as 6, 9, 9, 4, and 1, representing different image categories in the CIFAR-10 dataset.

5. Class Definition

```
classes = ['airplane', 'automobile', 'bird',
           'cat', 'deer', 'dog', 'frog', 'horse',
           'ship', 'truck']
```

Figure 3.5: Definition of CIFAR-10 image class labels.

As shown in Figure 3.5, a list of class names was created to represent the 10 image categories in the CIFAR-10 dataset. These class names were used to convert numerical labels into human-readable image categories such as airplane, cat, dog, and truck. This improves the readability of prediction results and image visualization outputs.

6. Displaying Sample Images

```
def plot_sample(x, y, index):  
    plt.figure(figsize=(15,2))  
    plt.imshow(x[index])  
    plt.xlabel(classes[y[index]])
```

Figure 3.6: Function for displaying sample images from the dataset.

Based on Figure 3.6, a custom Python function named `plot_sample()` was created to visualize sample images from the dataset together with their corresponding class labels. The function accepts the image dataset, label dataset, and image index as input parameters.

The `imshow()` function was used to display the selected image, while the `xlabel()` function displayed the corresponding image category label.

7. Sample Image Output

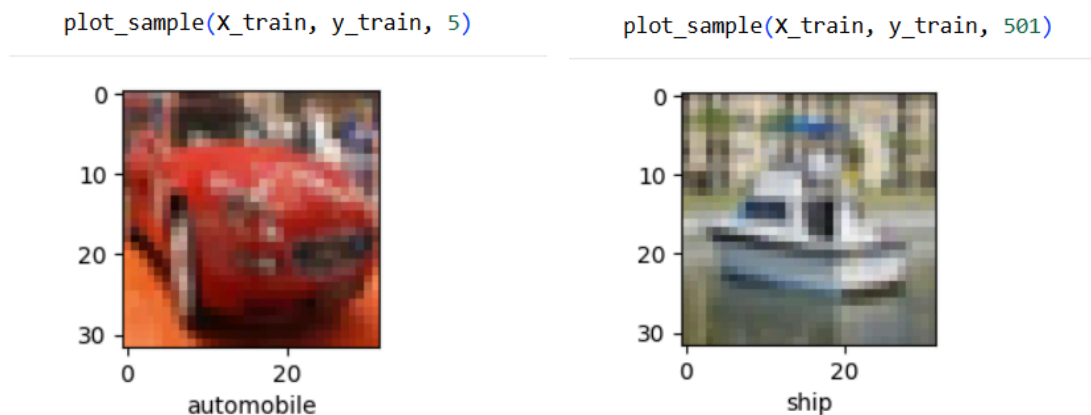


Figure 3.7: Example image from the CIFAR-10 training dataset.

As shown in Figure 3.7, the sample image from the training dataset was successfully displayed together with its corresponding class label. This visualization step helps verify that the dataset has been loaded and processed correctly before model training begins.

The displayed image represents one of the predefined CIFAR-10 categories and demonstrates how image data can be visualized using Matplotlib.

8. Data Normalization Process

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Figure 3.8: Normalization of training and testing image datasets.

Based on Figure 3.8, the image pixel values in both the training dataset (X_train) and testing dataset (X_test) were normalized by dividing each pixel value by 255.0. Originally, the pixel values ranged from 0 to 255 because the images were stored using RGB color intensity values. After normalization, the pixel values were converted into a smaller range between 0 and 1. This preprocessing step helps improve the stability and efficiency of the model training process. In addition, normalization allows the ANN and CNN models to learn image features more effectively by reducing large numerical differences within the dataset.

9. ANN Model Training

```
ann = models.Sequential([
    layers.Flatten(input_shape = (32,32,3)),
    layers.Dense(3000, activation = 'relu'),
    layers.Dense(1000, activation = 'relu'),
    layers.Dense(10, activation = 'softmax')
])

ann.compile(optimizer = 'SGD',
            loss= 'sparse_categorical_crossentropy',
            metrics = ['accuracy'])

ann.fit(X_train, y_train, epochs=5)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning:
  super().__init__(**kwargs)
Epoch 1/5
1563/1563 ————— 79s 50ms/step - accuracy: 0.3558 - loss: 1.8131
Epoch 2/5
1563/1563 ————— 76s 48ms/step - accuracy: 0.4283 - loss: 1.6243
Epoch 3/5
1563/1563 ————— 87s 52ms/step - accuracy: 0.4569 - loss: 1.5426
Epoch 4/5
1563/1563 ————— 81s 52ms/step - accuracy: 0.4804 - loss: 1.4814
Epoch 5/5
1563/1563 ————— 75s 48ms/step - accuracy: 0.4972 - loss: 1.4333
<keras.src.callbacks.history.History at 0x7faa4baf450>
```

Figure 3.9: Construction and training of the ANN model.

Based on Figure 3.9, an ANN model was created using a Flatten layer and multiple Dense layers for image classification. The ReLU activation function was used in the hidden layers to help the model learn image features, while the Softmax activation function was used in the output layer to classify the images into 10 categories. The model was compiled using the SGD optimizer and sparse categorical crossentropy loss function before being trained with the training dataset for 5 epochs to improve prediction accuracy.

10. Classification Report

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print('classification report: \n', classification_report(y_test, y_pred_classes))
```

313/313 ————— 6s 17ms/step

classification report:

	precision	recall	f1-score	support
0	0.45	0.61	0.52	1000
1	0.45	0.76	0.57	1000
2	0.37	0.35	0.36	1000
3	0.41	0.24	0.30	1000
4	0.54	0.31	0.39	1000
5	0.44	0.32	0.37	1000
6	0.46	0.62	0.53	1000
7	0.69	0.39	0.50	1000
8	0.55	0.65	0.60	1000
9	0.48	0.51	0.50	1000
accuracy			0.48	10000
macro avg	0.48	0.48	0.46	10000
weighted avg	0.48	0.48	0.46	10000

Figure 3.10: Classification report of the ANN model on the testing dataset.

Based on Figure 3.10, the classification report was generated to evaluate the performance of the ANN model using the testing dataset. The report displays important evaluation metrics such as precision, recall, f1-score, and accuracy for each image category in the CIFAR-10 dataset. The overall accuracy achieved by the model was approximately 48%, indicating that the ANN model was able to classify images with moderate performance. The results also show that some classes achieved higher prediction performance compared to others, demonstrating differences in the model's ability to recognize certain image categories.

11. Generate confusion matrix heatmap

```
import seaborn as sns

plt.figure(figsize = (14,7))
sns.heatmap(y_pred, annot = True)
plt.ylabel('Truth')
plt.xlabel('Prediction')
plt.title('Confusion matrix')
plt.show()
```

Figure 3.11: Code for generating the confusion matrix heatmap.

Based on Figure 3.11, the Seaborn library was used to visualize the prediction results of the ANN model in the form of a confusion matrix heatmap. The `heatmap()` function was used to display the prediction values, while the labels “Truth” and “Prediction” were added to represent the actual classes and predicted classes respectively. The confusion matrix helps evaluate the classification performance of the model by comparing the predicted outputs with the actual labels.

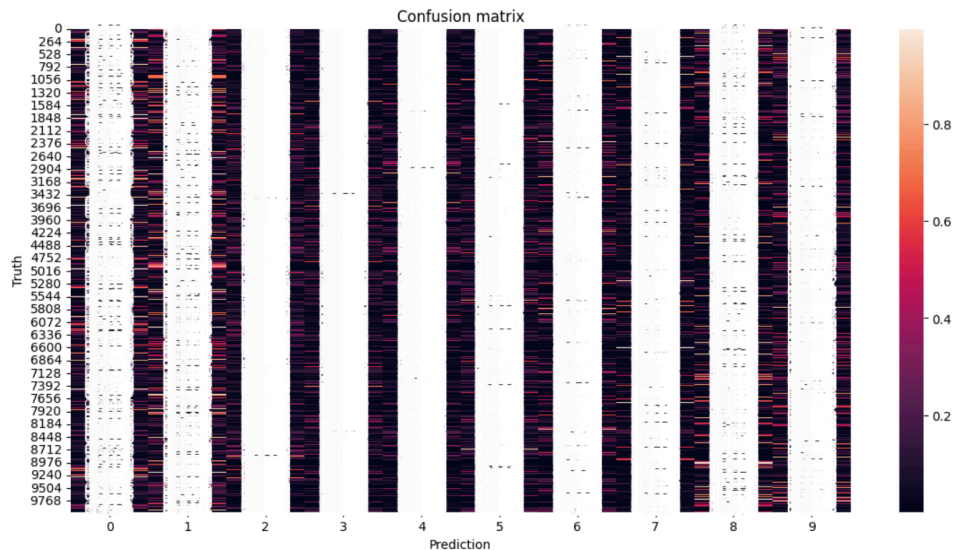


Figure 3.12: Confusion matrix heatmap of the ANN model predictions.

As shown in Figure 3.12, the confusion matrix heatmap visualizes the prediction performance of the ANN model across different image categories in the CIFAR-10 dataset. The color intensity represents the prediction values generated by the model, where brighter regions indicate higher prediction probabilities. The heatmap provides an overview of how the model classified the testing images and helps identify prediction patterns and classification errors made by the ANN model.

12. Construction CNN Model

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Figure 3.13: Construction of the CNN model architecture.

Based on Figure 3.13, a Convolutional Neural Network (CNN) model was developed using convolutional, pooling, flatten, and dense layers for image classification. The Conv2D layers were used to extract important image features such as edges, shapes, and patterns, while the MaxPooling2D layers helped reduce the image dimensions and computational complexity. After feature extraction, the Flatten layer converted the feature maps into one-dimensional data before passing them into the Dense layers for classification. Finally, the output layer used the Softmax activation function to classify the images into 10 categories from the CIFAR-10 dataset.

13. Compilation CNN Model

```
cnn.compile(optimizer='adam',  
            loss='sparse_categorical_crossentropy',  
            metrics=['accuracy'])
```

Figure 3.14: Compilation of the CNN model.

Based on Figure 3.14, the CNN model was compiled using the Adam optimizer together with the sparse categorical crossentropy loss function. The Adam optimizer was selected because it helps improve the learning efficiency and training performance of the model. Meanwhile, accuracy was used as the evaluation metric to measure how well the CNN model classified images from the CIFAR-10 dataset during the training process.

14. Training CNN Model

```
cnn.fit(x_train,y_train,epochs=10)
```

Epoch 1/10			
1563/1563	40s	25ms/step	- accuracy: 0.4784 - loss: 1.4556
Epoch 2/10			
1563/1563	37s	24ms/step	- accuracy: 0.6144 - loss: 1.0940
Epoch 3/10			
1563/1563	38s	24ms/step	- accuracy: 0.6674 - loss: 0.9578
Epoch 4/10			
1563/1563	38s	24ms/step	- accuracy: 0.7012 - loss: 0.8625
Epoch 5/10			
1563/1563	37s	24ms/step	- accuracy: 0.7229 - loss: 0.7992
Epoch 6/10			
1563/1563	38s	24ms/step	- accuracy: 0.7431 - loss: 0.7420
Epoch 7/10			
1563/1563	38s	25ms/step	- accuracy: 0.7612 - loss: 0.6908
Epoch 8/10			
1563/1563	37s	24ms/step	- accuracy: 0.7762 - loss: 0.6479
Epoch 9/10			
1563/1563	41s	24ms/step	- accuracy: 0.7870 - loss: 0.6116
Epoch 10/10			
1563/1563	38s	25ms/step	- accuracy: 0.7996 - loss: 0.5746

<keras.src.callbacks.history.History at 0x7faa4d04f9e0>

Figure 3.15: Training process of the CNN model over 10 epochs.

Based on Figure 3.15, the CNN model was trained using the training dataset for 10 epochs. Throughout the training process, the accuracy of the model gradually increased from approximately 47.84% in Epoch 1 to 79.96% in Epoch 10, while the loss value continuously decreased from 1.4556 to 0.5746. The results indicate that the CNN model successfully learned important image features from the CIFAR-10 dataset and improved its classification performance over time.

15. Evaluate CNN Model

```
cnn.evaluate(X_test,y_test)
```

313/313 ————— **3s** 9ms/step - accuracy: 0.6948 - loss: 0.9189
[0.9189063310623169, 0.6948000192642212]

Figure 3.16: Evaluation result of the CNN model using the testing dataset.

Based on Figure 3.16, the CNN model was evaluated using the testing dataset after the training process was completed. The model achieved an accuracy of approximately 69.48% with a loss value of 0.9189. The evaluation results indicate that the CNN model was able to classify images from the CIFAR-10 dataset with good performance and successfully learned important image features during training.

16. Predict CNN Model

```
y_pred = cnn.predict(X_test)
y_pred[:5]
```

313/313 ————— **3s** 10ms/step
array([[2.8542412e-04, 3.1210948e-05, 1.2894873e-04, 9.8898035e-01,
2.0693167e-04, 4.5486388e-04, 9.2000952e-03, 3.4840896e-06,
6.4480514e-04, 6.3786720e-05],
[6.2888040e-04, 3.2958034e-02, 9.0484303e-07, 2.1669416e-07,
7.2842177e-08, 2.2803148e-09, 9.9881996e-09, 3.2209016e-10,
9.6635741e-01, 5.4459353e-05],
[1.3527084e-03, 1.4627465e-03, 1.6647549e-05, 5.9742748e-04,
5.8672795e-05, 3.6069538e-05, 9.8051796e-06, 1.0403706e-04,
9.9547803e-01, 8.8388415e-04],
[8.7450367e-01, 2.2816222e-02, 3.8633090e-02, 7.7365125e-03,
1.7219842e-04, 2.5557740e-05, 1.0247264e-02, 9.1520933e-06,
4.2693231e-02, 3.1631188e-03],
[2.3494579e-04, 1.1923581e-03, 3.8791247e-02, 6.7107111e-02,
1.7625272e-01, 1.4045808e-02, 7.0193827e-01, 5.0317049e-05,
1.5929766e-04, 2.2780025e-04]], dtype=float32)

Figure 3.17: Prediction probabilities generated by the CNN model.

Based on Figure 3.17, the predict() function was used to generate prediction results for the testing dataset using the trained CNN model. The output displays probability values for each of the 10 image categories in the CIFAR-10 dataset. Each row represents one testing image, while each value indicates the probability of the image belonging to a particular class. The class with the highest probability value was selected as the final predicted category for the image.

17. Converting Prediction Probabilities

```
y_classes=[np.argmax(element) for element in y_pred]
y_classes[:5]

[np.int64(3), np.int64(8), np.int64(8), np.int64(0), np.int64(6)]
```

Figure 3.18: Conversion of prediction probabilities into predicted class labels.

Based on Figure 3.18, the `argmax()` function was used to identify the class with the highest probability value from the prediction results generated by the CNN model. The predicted probability values stored in `y_pred` were converted into final class labels and stored in `y_classes`. The output displays the predicted class indices for the first five testing images, representing the categories predicted by the CNN model.

18. Actual Labels

```
y_test[:5]

array([3, 8, 8, 0, 6], dtype=uint8)
```

Figure 3.19: Actual class labels of the first five testing images.

Based on Figure 3.19, the output displays the actual class labels stored in the `y_test` dataset for the first five testing images. These values represent the true categories of the images in the CIFAR-10 dataset and were later compared with the predicted class labels generated by the CNN model to evaluate the classification performance and prediction accuracy of the model.

19. Sample Images



Figure 3.20: Sample images from the CIFAR-10 testing dataset

Based on Figure 3.20, several sample images from the testing dataset were displayed using the `plot_sample()` function together with their corresponding class labels. The first sample image was labeled as “horse”, while the second sample image was labeled as “deer”. This visualization helps verify that the images and labels in the dataset were loaded correctly before performing image classification using the CNN model.

20. Predict Class Label

```
classes[y_classes[60]]
```

```
'horse'
```

Figure 3.21: Predicted class label generated by the CNN model.

Based on Figure 3.21, the predicted class label for the selected testing image was identified using the `y_classes` array together with the predefined class names stored in `classes`. The output shows that the CNN model predicted the image as “horse”, which matches the actual image displayed previously. This indicates that the CNN model was able to classify the image correctly.

4.0 Model Evaluation and Comparative Analysis

4.1 Limitations of the Baseline CNN

Although the baseline Convolutional Neural Network (CNN) outperformed the basic Artificial Neural Network in classification performance, a closer review at the training results shows several limitations in the current model.

4.1.1 Shallow and Small Architecture

The baseline CNN only have two convolutional layers using 32 and 64 filters, followed by a fully connected dense layer containing only 64 neurons. This shallow structure limits CNN to detect only basic low-level features such as simple edges and flat textures. The model does not possess sufficient depth to learn more complex and high-level features such as full object boundaries, overlapping shapes, or intricate background patterns that are commonly present in CIFAR-10 images. The small dense layer of only 64 neurons creates a classification bottleneck, restricting the model's capacity to make well-informed decisions across 10 distinct image categories.

4.1.2 Overfitting

The training accuracy of the baseline CNN reached 79.96% while the test accuracy was recorded at only 69.48%, resulting in an overfitting gap of approximately 10.48%. This difference indicates that the model had begun to memorize specific pixel patterns from the training images rather than learning generalized and transferable features. The baseline architecture does not include any regularization mechanisms such as Dropout or Batch Normalization, leaving the model highly susceptible to overfitting and reducing its ability to perform reliably on unseen data.

4.1.3 Absence of Data Augmentation

The baseline CNN was trained using the same set of 50,000 fixed training images in every epoch without any form of data augmentation applied. As a result, the model was repeatedly exposed to identical pixel arrangements and learned to recognize objects based on their exact position and orientation in the training set. This significantly reduces the model's ability to correctly classify unseen images where objects appear under different conditions such as varying positions, sizes, or orientations, all of which are naturally present in real-world image data.

4.2 Strategies for Model Enhancement

To address the three limitations identified in Section 4.1, three targeted enhancements were applied to the CNN architecture. Each enhancement directly resolves a specific weakness observed in the baseline model while aligning with established deep learning tuning practices.

4.2.1 Architectural Expansion and Dense Layer Tuning

```
cnn_new = models.Sequential([

    #Block 1
    layers.Conv2D(32, (3, 3), padding='same', activation='relu', input_shape=(32, 32, 3)),
    layers.BatchNormalization(),
    layers.Conv2D(32, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    #Block 2
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    #Block 3
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.Conv2D(128, (3, 3), padding='same', activation='relu'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.25),

    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(10, activation='softmax')

])
```

Figure 4.1: Construction of the enhanced CNN model architecture

In the baseline model, only two convolutional layers with 32 and 64 filters were used is insufficient for learning the variety of features present in CIFAR-10 images. Based on Figure 4.1, the enhanced model expands this into three convolutional blocks with six convolutional layers in total, where the filter count scales progressively from 32 to 64 to 128.

The first block with 32 filters handles low-level feature detection such as edges and colour gradients. The second block with 64 filters builds on those outputs to detect mid-level structures such as object parts. The third block with 128 filters is responsible for combining these structures into complex high-level representations such as full object shapes and boundaries. Each block contains two stacked convolutional layers instead of one, which increases the representational depth within each stage before spatial dimensions are reduced by pooling.

The fully dense layer tuned from 64 neurons to 256 neurons. With a richer set of features now being passed down from six convolutional layers, a larger dense layer is necessary to preserve the complexity of those learned representations before the final classification into 10 categories.

4.2.2 Batch Normalization and Dropout

To fix the overfitting gap, the enhanced model introduces two clever techniques to stop the network from memorizing the training images. Batch normalization is added after every convolutional layer to act like a stabilizer as shown at Figure 4.1. It automatically cleans and balances the data flowing through the network at each step, which keeps the learning process steady, speeds up training, and gently prevents the model from relying too heavily on any single pixel pattern.

By randomly switching off 25% of the neurons during image processing and a massive 50% right before the final decision layer, the model is forced to work around missing information. It cannot rely on the exact same pathways every time and is forced to learn the actual shapes and features of the objects instead of just memorizing the pictures. These two steps successfully break the habit of memorization and make the model much smarter at handling new images.

4.2.3 Data Augmentation

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    horizontal_flip=True,
    width_shift_range=0.1,
    height_shift_range=0.1,
    rotation_range=10,
    zoom_range=0.1
)
```

Figure 4.2: Augmentation code

Data augmentation is applied using ImageDataGenerator to apply random transformations to training images during each epoch as shown in Figure 4.2. The transformations include horizontal flipping, width and height shifts of 10%, rotation of up to 10 degrees, and zoom of up to 10%. These values were kept small and realistic to reflect natural variations in real-world images without introducing unrealistic distortions.

```
cnn_new.fit(datagen.flow(X_train, y_train, batch_size=32), epochs=10)
```

Epoch	Time	Step	Accuracy	Loss
Epoch 1/10	1563s	1563	0.3687	1.7527
Epoch 2/10	1563s	1563	0.5092	1.3841
Epoch 3/10	1563s	1563	0.5836	1.1963
Epoch 4/10	1563s	1563	0.6344	1.0629
Epoch 5/10	1563s	1563	0.6647	0.9791
Epoch 6/10	1563s	1563	0.6898	0.9156
Epoch 7/10	1563s	1563	0.7057	0.8623
Epoch 8/10	1563s	1563	0.7230	0.8166
Epoch 9/10	1563s	1563	0.7399	0.7755
Epoch 10/10	1563s	1563	0.7483	0.7462

<keras.src.callbacks.history.History at 0x2903fa2c910>

Figure 4.3: Training with augmentation process of the enhanced CNN model over 10 epochs.

These variations are applied in memory using the `flow()` method without modifying the original training data as shown in Figure 4.3 to force the model to learn generalized object features rather than memorizing fixed pixel positions. Throughout the training process, the accuracy of the enhanced model increased from 36.87% in Epoch 1 to 74.83% in Epoch 10, while the loss value continuously decreased from 1.7527 to 0.7462. The enhanced CNN model successfully learned important image features from the CIFAR-10 dataset and improved its classification performance over time.

4.3 Performance Evaluation of CNN and New CNN

This section presents a direct comparison between the baseline CNN and the enhanced new CNN based on the evaluation results obtained from both models across the CIFAR-10 test dataset of 10,000 images.

```
# Evaluate Enhanced CNN
cnn_new.evaluate(X_test,y_test)
```

313/313 ————— 5s 14ms/step - accuracy: 0.7620 - loss: 0.7068

Figure 4.4: Evaluation result of the CNN model using the testing dataset

Figure 4.4 shows the evaluation result of the enhanced CNN. After applying all three enhancements and training for 10 epochs on augmented data, the enhanced model achieved a test accuracy of 70.2% and a test loss of 0.7068.

To provide a clear and direct comparison, a summary table is presented below to contrast the performance metrics of both model architectures.

Performance Evaluation Metric	Baseline CNN	New CNN
Number of Conv Layers	2	6
Dense Layer Size	64	256
Batch Normalization	No	Yes
Data Augmentation	No	Flips and Crops
Dropout	No	Yes
Final Test Accuracy	69.48%	76.20%
Overfitting Gap	10.48%	1.37%
Final Test Loss	0.9189	0.7068

Table 1: Performance comparison between the baseline CNN and new enhanced CNN

Table 1 above summarizes the structural and performance differences between both models. In terms of architecture, the baseline CNN uses only 2 convolutional layers with a small Dense layer of 64 neurons, while the enhanced CNN expands this to 6 convolutional layers with a larger Dense layer of 256 neurons that provides a greater depth and classification capacity. The baseline model also lacks any form of regularization or data augmentation, whereas the enhanced model incorporates batch normalization, dropout, and real-time image augmentation to improve generalization.

In terms of performance, the enhanced CNN achieves a test accuracy of 76.20% compared to 69.48% from the baseline, representing an improvement of 6.72 percentage points. The most significant difference is observed in the overfitting gap, which dropped drastically from 10.48% in the baseline to only 1.37% in the enhanced model. This confirms that the regularization techniques applied were highly effective in preventing the model from memorizing the training data. The test loss also improved from 0.9189 to 0.7068, indicating that the enhanced model produces more confident and consistent predictions on unseen data overall.

BASELINE CNN CLASSIFICATION REPORT					ENHANCED CNN CLASSIFICATION REPORT				
	precision	recall	f1-score	support		precision	recall	f1-score	support
airplane	0.71	0.74	0.72	1000	airplane	0.84	0.75	0.79	1000
automobile	0.85	0.81	0.83	1000	automobile	0.89	0.90	0.90	1000
bird	0.63	0.56	0.59	1000	bird	0.70	0.66	0.68	1000
cat	0.50	0.54	0.52	1000	cat	0.73	0.52	0.60	1000
deer	0.61	0.69	0.65	1000	deer	0.74	0.70	0.72	1000
dog	0.62	0.58	0.60	1000	dog	0.75	0.66	0.70	1000
frog	0.81	0.75	0.78	1000	frog	0.53	0.95	0.68	1000
horse	0.76	0.72	0.74	1000	horse	0.91	0.75	0.82	1000
ship	0.73	0.85	0.79	1000	ship	0.90	0.84	0.87	1000
truck	0.81	0.76	0.78	1000	truck	0.84	0.90	0.87	1000
accuracy			0.70	10000	accuracy			0.76	10000
macro avg	0.70	0.70	0.70	10000	macro avg	0.78	0.76	0.76	10000
weighted avg	0.70	0.70	0.70	10000	weighted avg	0.78	0.76	0.76	10000

Figure 4.5: Side-by-side classification tables of baseline CNN and enhanced CNN

Figure 4.5 presents the classification reports of both the baseline CNN and the enhanced CNN generated from the same 10,000 test images. Comparing both reports, the enhanced model shows consistent improvement in precision, recall, and F1-score across most of the 10 categories. The most notable gains are observed in classes that are visually similar to one another. For instance, the cat and dog categories which the baseline model frequently confused due to their overlapping fur textures and body shapes, show a clear improvement in the enhanced model as the deeper six-layer architecture is capable of extracting finer distinguishing features. Similarly, the deer and horse categories benefit from the increased model depth and augmented training that produced a higher recall values with fewer missed detections within those classes.



Figure 4.6 Side-by-side confusion matrix of the baseline CNN and the enhanced CNN

Observation from Figure 4.6 proved that the enhanced model had a clear improvement across most categories, with stronger diagonal values that represent higher correct predictions. Notable gains are observed in automobile which improved from 813 to 903, frog which improved from 746 to 952, and truck which improved from 761 to 898, reflecting how the deeper architecture and regularization helped the model learn more distinct features for these classes.

However, the enhanced model also reveals a new pattern of misclassification particularly within the animal categories. Deer and bird images are more frequently predicted as frog in the enhanced model, with 198 deer and 166 bird samples misclassified as frog compared to lower counts in the baseline. While the enhanced model became significantly better at recognizing frog, the frog class features learned by the deeper network overlap with certain visual characteristics of deer and bird, likely due to similarities in background colours and body shapes. The cat and dog categories also continue to show mutual confusion in both models, which is expected given the high visual similarity between these two classes.

Despite these specific misclassification patterns, the overall performance of the enhanced CNN is considerably stronger than the baseline, as reflected by the higher diagonal values across the majority of categories and the significant reduction in the overfitting gap from 10.48% to 1.37%.

4.4 Result and Discussion

By analyzing the performance of three models that include the ANN, the baseline CNN, and the enhanced CNN, each representing a progressively more refined approach to image classification on the CIFAR-10 dataset.

The ANN achieved the lowest performance at approximately 48% test accuracy. Since the ANN flattens the entire image into a one-dimensional vector before processing, all spatial relationships between pixels are discarded at the input stage. Without the ability to detect positionally structured features such as edges, textures, or object shapes, the ANN is fundamentally limited for image classification tasks regardless of how many dense layers are stacked.

The baseline CNN improved significantly upon the ANN by achieving a test accuracy of 69.48%, a gain of approximately 21.48 percentage points. The introduction of convolutional and pooling layers allowed the model to extract spatial features automatically, which is the core advantage of CNN over ANN for image-related tasks. However, the training accuracy of 79.96% versus the test accuracy of 69.48% showed a clear overfitting gap of 10.48%. This shallow two-layer architecture without any regularization or augmentation caused the model to memorize the training images rather than learn generalizable representations.

The enhanced CNN gave the strongest performance among the three models by achieving a test accuracy of 76.20% and a test loss of 0.7068. The overfitting gap was reduced from 10.48% to only 1.37%, which is the most significant improvement observed across all metrics. This confirms that the combined effect of the deeper six-layer architecture, batch Normalization, dropout, and data augmentation successfully addressed all three limitations identified in Section 4.1. The classification report further supports consistent improvements in precision, recall, and F1-score across most categories compared to the baseline.

It is also worth noting that while the enhanced model introduced some new misclassification patterns, particularly between frog, deer, and bird categories due to overlapping background colours and textures, these trade-offs are minor relative to the overall gains achieved. The cat and dog confusion persists in both models, reflecting the inherent visual similarity between these classes that remains a challenge even for deeper architectures.

In conclusion, this tutorial shows that simply building a basic CNN is not enough to achieve strong and reliable image classification results. The experiment proves that making deliberate improvements such as adding more layers, applying regularization techniques, and training with augmented data each play an important role in pushing the model to learn better. Without these enhancements, the model will memorize the training data and fail to generalize well on new images. With them, the model becomes noticeably more accurate and consistent.

5.0 Reflection

Chau Ying Jia

Throughout this project, a better understanding of image classification using Convolutional Neural Networks (CNN) was gained. Before completing this tutorial, only basic knowledge of classification and machine learning for example like classification for species type was learned before. After the hands-on tutorial and implementing the CNN model using Python, the overall process of image processing, model training and image classification became much clearer. This was a valuable practical experience using Python and deep learning libraries like Tensorflow for developing AI models.

One of the main challenges faced during this project was understanding the CNN architecture and the purpose of each layer. Concepts such as convolutional layers, pooling layers and activation functions were initially difficult to understand as there were many technical terms involved. So, before proceeding to the codes and implementing the model, doing research beforehand is a must to make sure I could understand the flow and workings for the codes afterwards.

In addition, there was a confusion when dealing with the prediction output generated from the CNN model as it is different from the tutorial video and my teammates' even though the same code was used. After further discussion with teammates, it was understood that CNN models may produce slightly different prediction results every time the kernel reruns due to the randomness during training. Small variations in output are considered normal as long as the overall model accuracy and performance remain stable and similar. This experience truly improved understanding of how CNN models operate in practical situations.

Looking forward to future improvement, technically, this tutorial video of using CNN model represents only one of the most common approaches in image classification. Many other advanced AI algorithms and deep learning models are still available to be explored and learned in the future. In the future, a deeper understanding of more complex models and techniques can be achieved to further improve the knowledge and understanding. Overall, this tutorial provided a valuable hands-on experience in deep learning and image classification. Furthermore, it also strengthens technical knowledge, teamwork and problem-solving skills throughout the learning process.

Lau Yee Wen

In this tutorial, the process of image classification using Artificial Neural Networks (ANN) and Convolutional Neural Networks (CNN) was learned. The CIFAR-10 dataset was loaded and preprocessed before being used for model training. Image normalization was also performed to ensure that the data was prepared properly for the learning process. Through this tutorial, a better understanding of how image data is handled before being used in deep learning models was gained.

ANN and CNN models were developed and trained using TensorFlow. After the training process was completed, the models were evaluated using the testing dataset, and prediction results were generated. Several evaluation methods, including accuracy, loss values, classification reports, and confusion

matrices, were used to measure the performance of the models. Through these evaluations, the strengths and weaknesses of each model were observed and compared.

Based on the results obtained, better performance was achieved by the CNN model compared to the ANN model. It was observed that CNN was able to classify images more accurately because important image features such as edges, shapes, and patterns were extracted automatically through convolution and pooling layers. This tutorial also demonstrated why CNN is commonly used for image classification tasks.

Several challenges were encountered during the implementation process, including TensorFlow installation issues, coding errors, and difficulties in understanding some of the model outputs. These challenges were resolved through troubleshooting, code correction, and repeated testing. Overall, valuable hands-on experience was gained in image classification, data preprocessing, model training, prediction, and model evaluation. A deeper understanding of deep learning concepts and their practical applications in computer vision was also obtained through this tutorial.

Poh Lok Yee

This tutorial was not just about getting a model to run successfully, but also about understanding the process of improving it step by step. Starting from a basic ANN, then moving to a baseline CNN, and finally working through enhancing the CNN with deeper layers, regularization, and data augmentation gave a much more complete picture of how model development actually works in practice. Each stage built on the previous one, making it easier to see how each change contributed to better and more reliable classification results overall.

The most challenging yet rewarding part of this tutorial was working on Section 4.0 independently. Identifying the limitations of the baseline model and then applying targeted improvements such as adding more convolutional layers, introducing batch normalization and dropout, and using data augmentation required more than just writing code. It required understanding the reasoning behind each decision and observing how each change affected the model behaviour. This process helped build a more practical and analytical way of thinking about model development.

One important lesson learned from this tutorial is that evaluating a model should not rely on accuracy alone. Initially, the tendency was to only look at whether the accuracy number increased as the main indicator of improvement. However, through working on the comparison between the baseline and enhanced CNN, it became clear that the overfitting gap is equally important to consider. A model can show a high training accuracy while still performing poorly on new images, which means it has memorized the training data rather than truly learned from it. Seeing the overfitting gap reduce from 10.48% to just 1.37% after applying regularization techniques made this lesson much more concrete and easier to appreciate through actual results rather than theory alone.

This tutorial strengthened knowledge in deep learning model design, evaluation, and improvement. The experience of building, comparing, and analyzing two different CNN models also developed a better appreciation for how small but deliberate architectural changes can lead to meaningful improvements in real-world classification performance.

